

OrderManagement

Documentação Técnica Completa

API REST em ASP.NET Core + Frontend em React

Vitor Miranda Jeremias
Estudante de Engenharia de Software - PUCPR
github.com/Vitormjere

Sumário

1. Visão Geral
2. Arquitetura Geral
3. Backend - OrderManagementAPI
4. Frontend - OrderManagementFrontend
5. Deploy e CI/CD
6. Desafios Reais e Como Foram Resolvidos
7. Como Rodar os Projetos Localmente

1. Visão Geral

O OrderManagement é um sistema full-stack para gerenciamento de pedidos de uma loja. Ele é composto por uma API REST desenvolvida em C# (backend) e uma aplicação em React (frontend), que se comunicam para realizar todas as operações do sistema.

O projeto vai além de um CRUD básico. Além do cadastro de produtos, clientes e pedidos, ele implementa regras de negócio que seriam esperadas em uma aplicação real. Ao criar um pedido, por exemplo, o sistema verifica se há estoque suficiente, reduz automaticamente a quantidade disponível e registra o preço do produto naquele momento, evitando que alterações futuras afetem pedidos já realizados

Links

- API em produção: ordermanagement-api-vitor-fpcphmf5ejaretc0.brazilsouth-01.azurewebsites.net/swagger
- Frontend em produção: order-management-frontend-tan.vercel.app
- Repositório backend: github.com/Vitormjere/OrderManagementAPI
- Repositório frontend: github.com/Vitormjere/OrderManagementFrontend

Stack utilizada

Camada	Tecnologias
Backend	ASP.NET Core 8, Entity Framework Core, SQL Server / Azure SQL
Frontend	React (Vite), React Router, CSS puro
Infraestrutura	Azure App Service, Azure SQL Database, Vercel
CI/CD	GitHub Actions (backend), deploy automático Vercel (frontend)

2. Arquitetura Geral

O fluxo de uma requisição típica segue este caminho:

Navegador (React)

|

| fetch() → requisição HTTP (JSON)

▼

API ASP.NET Core (Controllers)

|

|─ Validação dos dados (DTOs + Data Annotations)

▼

Entity Framework Core (AppDbContext)

|

|─ Converte as operações para SQL

▼

Azure SQL Database

|

|─ Executa a consulta e retorna os dados

▼

Resposta JSON

|

▼

React atualiza a interface

No backend, o projeto foi organizado em camadas para separar melhor as responsabilidades. Os Controllers recebem as requisições da API, os Models representam as entidades do sistema, os DTOs definem os dados enviados e recebidos pela API, a pasta Data concentra o acesso ao banco de dados usando Entity Framework Core e o Middleware centraliza o tratamento de erros.

No frontend, a organização também foi feita para manter o código simples de navegar. As pages representam cada tela da aplicação, services reúne as funções responsáveis pela comunicação com a API e o React Router controla a navegação entre as páginas de acordo com a URL.

3. Backend - OrderManagementAPI

3.1 Modelagem de dados

O domínio da aplicação é composto por quatro entidades: Product, Customer, Order e OrderItem. Um Order pertence a um Customer e pode conter vários OrderItems, enquanto cada OrderItem está associado a um único Product.

Uma decisão importante foi armazenar o preço do produto no próprio OrderItem no momento em que o pedido é criado. Dessa forma, se o preço de um produto mudar no futuro, os pedidos antigos continuam exibindo o valor que foi pago na época da compra.

3.2 Decisões técnicas e por quê

Controllers em vez de Minimal APIs

Escolha: Organizei os endpoints utilizando Controllers (ApiController), separando cada recurso em sua própria classe.

Por quê: Preferi essa abordagem porque ela deixa o projeto mais organizado e facilita separar as responsabilidades. Além disso, Controllers são o padrão mais comum em aplicações ASP.NET Core e foi uma boa oportunidade para praticar uma estrutura parecida com a utilizada em projetos profissionais.

Alternativa considerada: Utilizar Minimal APIs. Elas exigem menos código e funcionam muito bem em projetos menores, mas para este projeto achei que os Controllers deixariam a organização mais clara

Entity Framework Core (Code-First)

Escolha: Utilizei a abordagem Code First, onde as classes C# definem a estrutura da aplicação e o banco é criado por meio das Migrations.

Por quê: Além de evitar escrever SQL manualmente para criar as tabelas, essa abordagem facilitou bastante a evolução do projeto. Conforme novas entidades e relacionamentos foram surgindo, bastava criar uma nova Migration para manter o banco atualizado, tanto no ambiente local quanto no Azure SQL.

Alternativa considerada: Database First ou Dapper. Ambas funcionariam, mas para esse projeto o Code First oferecia uma produtividade maior e uma integração melhor com o Entity Framework Core

DTOs para entrada e saída

Escolha: Todas as requisições e respostas da API utilizam DTOs. As entidades do Entity Framework nunca são expostas diretamente.

Por quê: Fiz isso principalmente por segurança. Assim, o cliente da API não consegue enviar campos que deveriam ser controlados pelo servidor, como o Id de um recurso ou o preço de um produto. Além disso, essa separação resolveu um problema de referência circular durante a serialização do JSON, que encontrei enquanto desenvolvia o projeto.

Alternativa considerada: Expor as entidades diretamente. Embora fosse mais rápido de implementar, essa abordagem acabou gerando erros de serialização e deixaria a API mais exposta a alterações internas do projeto

Middleware global de tratamento de erros

Escolha: Criei um middleware responsável por capturar exceções não tratadas e retornar respostas JSON padronizadas.

Por quê: Em vez de colocar blocos try/catch em praticamente todos os Controllers, preferi concentrar esse tratamento em um único lugar. Além de deixar o código mais limpo, isso evita que detalhes internos da aplicação, como stack traces, sejam enviados para quem consome a API.

Alternativa considerada: Tratar exceções dentro de cada endpoint. Apesar de funcionar, essa solução gera bastante repetição de código e aumenta a chance de algum erro passar despercebido

Retry-on-failure na conexão com o banco

Escolha: Configurei o `EnableRetryOnFailure` no Entity Framework Core para que a aplicação tente se reconectar automaticamente ao banco em caso de falha.

Por quê: Durante o desenvolvimento percebi que o Azure SQL Database, no plano Serverless, entra em hibernação depois de um tempo sem uso. Quando isso acontece, a primeira conexão normalmente falha com o erro **40613**. Em vez de devolver um erro para o usuário logo na primeira tentativa, configurei o Entity Framework para tentar novamente automaticamente. Isso resolveu um problema que encontrei várias vezes durante os testes e também depois do deploy.

Alternativa considerada: Não utilizar retry automático. Nesse caso, sempre que o banco estivesse "dormindo", a primeira requisição retornaria erro e o usuário teria que tentar novamente manualmente.

3.3 Exemplo de código - criação de um pedido

O método abaixo concentra a principal regra de negócio da aplicação: antes de criar um pedido, o sistema verifica se existe estoque suficiente para cada produto. Caso algum item não tenha quantidade disponível, a operação é interrompida e nenhum dado é salvo no banco

```
foreach (var itemDto in dto.Items)
{
    var product = await _context.Products.FindAsync(itemDto.ProductId);

    if (product.StockQuantity < itemDto.Quantity)
        return BadRequest($"Insufficient stock for '{product.Name}'.");

    product.StockQuantity -= itemDto.Quantity;

    order.Items.Add(new OrderItem {
        ProductId = product.Id,
        Quantity = itemDto.Quantity,
        UnitPrice = product.Price // preço "congelado"
    });
}

await _context.SaveChangesAsync(); // salva pedido + estoque juntos
```

O pedido e a atualização do estoque são enviados ao banco em uma única chamada ao `SaveChangesAsync()`. Dessa forma, todas as alterações são persistidas juntas, garantindo que o pedido só seja criado se toda a validação for concluída com sucesso

3.4 Validação de dados

Os DTOs utilizam Data Annotations para validar automaticamente os dados recebidos pela API. Como os Controllers possuem o atributo `[ApiController]`, o ASP.NET Core verifica essas validações antes mesmo de executar o método. Se algum dado for inválido, a própria framework retorna uma resposta 400 Bad Request, sem necessidade de escrever código extra para isso

```
public class CreateCustomerDto
{
    [Required, StringLength(100, MinimumLength = 2)]
    public string Name { get; set; }

    [Required, EmailAddress]
    public string Email { get; set; }
}
```

3.5 Endpoints disponíveis

Recurso	Endpoints
Products	GET / POST /api/products GET / PUT / DELETE /api/products/{id}
Customers	GET / POST /api/customers GET / PUT / DELETE /api/customers/{id}
Orders	GET / POST /api/orders GET / DELETE /api/orders/{id} PUT /api/orders/{id}/status

4. Frontend - OrderManagementFrontend

4.1 Decisões técnicas e por quê

React com Vite

Escolha: O projeto foi criado utilizando o Vite.

Por quê: Além de ser o padrão atual para aplicações React, ele oferece uma inicialização muito mais rápida e um tempo de atualização menor durante o desenvolvimento. Isso tornou o trabalho mais ágil conforme o projeto foi crescendo.

Alternativa considerada: Create React App. Apesar de ainda funcionar, ele já não é a principal escolha da comunidade e apresenta um processo de desenvolvimento mais lento

CSS puro, sem framework

Escolha: Toda a estilização foi feita utilizando CSS puro.

Por quê: Optei por essa abordagem para praticar melhor CSS e entender exatamente como cada propriedade afeta a interface. Como o objetivo era aprendizado, preferi não utilizar bibliotecas que abstraíssem esse trabalho.

Alternativa considerada: Tailwind CSS ou Bootstrap. Essas ferramentas aceleram bastante o desenvolvimento, mas neste projeto preferi construir toda a interface manualmente

React Router para navegação

Escolha: A navegação da aplicação foi implementada com React Router.

Por quê: Como a aplicação possui várias páginas, o React Router permite navegar entre elas sem recarregar a página inteira, oferecendo uma experiência muito mais próxima de uma aplicação desktop.

Alternativa considerada: Controlar a navegação manualmente utilizando estados. Embora fosse possível, essa solução não escala bem e não mantém a URL sincronizada com a tela exibida.

fetch nativo em vez de Axios

Escolha: Todas as chamadas HTTP utilizam a API nativa fetch.

Por quê: Para o tamanho deste projeto, o fetch já atendia todas as necessidades. Preferi evitar adicionar mais uma dependência quando o navegador já oferece uma solução suficiente.

Alternativa considerada: Axios. Ele possui alguns recursos extras, como interceptadores, mas que não eram necessários para este projeto

4.2 Padrão de comunicação com a API

Todas as chamadas HTTP são centralizadas em um único arquivo, `services/api.js`:

```
const API_BASE_URL = "https://.../api";

export async function getProducts() {
  const response = await fetch(`${API_BASE_URL}/products`);
  if (!response.ok) throw new Error("Erro ao buscar produtos");
  return response.json();
}
```

Centralizar essas funções evita repetir a URL base em vários arquivos e facilita alterar o ambiente da aplicação, como trocar entre a API local e a publicada no Azure

4.3 Formulários controlados

Os formulários utilizam componentes controlados, ou seja, os valores dos campos ficam armazenados no estado do React

```
const [name, setName] = useState('')

<input
  value={name}
  onChange={(e) => setName(e.target.value)}
/>
```

4.4 Lista dinâmica de itens (tela de Pedidos)

A tela de Orders permite adicionar múltiplos itens antes de enviar o pedido. Isso é feito mantendo um array de objetos no estado, sempre criando uma **nova** cópia do array a cada alteração (nunca modificando o array original diretamente) — prática essencial em React para que a interface seja re-renderizada corretamente.

```
function addItem() {
  setItems([...items, { productId: '', quantity: 1 }])
}
```

5. Deploy e CI/CD

5.1 Backend — Azure App Service

A API está hospedada no Azure App Service, utilizando o plano gratuito, enquanto o banco de dados roda no Azure SQL Database Serverless. Todo o processo de deploy é automático: sempre que faço um push para a branch principal, o GitHub Actions compila o projeto, publica a API e realiza o deploy no Azure

```
jobs:
  build:
    steps:
      - run: dotnet build --configuration Release
      - run: dotnet publish --output ./publish
  deploy:
    needs: build
    steps:
      - uses: azure/webapps-deploy@v2
        with:
          app-name: ordermanagement-api-vitor
          publish-profile: ${ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }
```

Para autenticar o workflow, utilizei um Publish Profile, armazenado como um Secret no GitHub. Inicialmente tentei utilizar uma Service Principal, mas a conta institucional que estava usando não possuía permissão para criar as credenciais necessárias. O Publish Profile resolveu esse problema sem impactar o processo de deploy

5.2 Frontend — Vercel

O frontend foi publicado na Vercel, que também realiza o deploy automaticamente sempre que há alterações no repositório.

Como a aplicação utiliza o React Router, foi necessário criar um arquivo `vercel.json` para que qualquer rota seja redirecionada para o `index.html`. Sem essa configuração, ao atualizar uma página como `/products`, a Vercel retornava erro 404 por tentar localizar um arquivo físico com esse nome

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

6. Desafios Reais e Como Foram Resolvidos

Esta seção documenta problemas reais enfrentados durante o desenvolvimento — úteis para revisão antes de entrevistas técnicas, pois mostram raciocínio de depuração aplicado.

Referência circular na serialização JSON

Problema: Ao retornar um Order com Include(Customer) e Include(Items), a serialização entrava em loop infinito (Order → Customer → Orders → Customer...), gerando erro 500.

Solução: Criação de DTOs de resposta (OrderResponseDto, CustomerSummaryDto) que não incluem a lista de pedidos dentro do cliente, quebrando o ciclo.

Banco de dados hibernado (erro 40613)

Problema: O Azure SQL Database serverless entra em hibernação após inatividade; a primeira query após esse período falhava com 'Database is not currently available'.

Solução: Configuração de EnableRetryOnFailure no EF Core, fazendo a aplicação tentar reconectar automaticamente antes de reportar erro.

CORS bloqueando o frontend

Problema: O navegador bloqueava as requisições do React para a API por padrão, por serem origens diferentes.

Solução: Configuração explícita de CORS no Program.cs, liberando tanto localhost:5173 (desenvolvimento) quanto o domínio da Vercel (produção).

404 ao recarregar rotas internas na Vercel

Problema: Ao acessar diretamente uma rota como /products e recarregar a página, a Vercel retornava 404 por procurar um arquivo físico com esse nome.

Solução: Arquivo vercel.json com regra de rewrite, redirecionando qualquer rota para index.html e deixando o React Router assumir o roteamento no navegador.

Falha ao gerar credenciais automáticas no GitHub Actions

Problema: O Visual Studio tentou criar uma Service Principal automaticamente para autenticar o deploy, mas falhou por falta de permissão de administrador na conta institucional (Azure AD).

Solução: Uso de Publish Profile como método alternativo de autenticação, configurado manualmente como Secret no repositório GitHub.

7. Como Rodar os Projetos Localmente

7.1 Backend

```
git clone https://github.com/Vitormjere/OrderManagementAPI.git
cd OrderManagementAPI/OrderManagementAPI
dotnet restore
dotnet ef database update
dotnet run
```

Requer .NET 8 SDK e SQL Server LocalDB. Ajustar a connection string em appsettings.json.

7.2 Frontend

```
git clone https://github.com/Vitormjere/OrderManagementFrontend.git
cd OrderManagementFrontend
npm install
npm run dev
```

Requer Node.js. Por padrão aponta para a API de produção; para usar uma API local, ajustar API_BASE_URL em src/services/api.js.